Burrows–Wheeler Transform

Rearrange the characters to increase possibility of character runs.

`banana$` → BWT → `annb$aa`

Same characters. Different order.

The algorithm is fully reversible, no information lost.

Why rearrange a text?

Original

banana$

After BWT

annb$aa

Similar preceding characters often cluster.

This makes the data better suited for compression methods such as run-length encoding and entropy encoding.

BWT exploits the fact that patterns repeat in text

Sorting cyclic rotations places similar following contexts next to each other.

The last column contains the character immediately before each context.

Example: In english “t” often precedes “he” and “i” often precedes “ng” (“the” and “shopping”)

Exploits structure and repetition in text

Forward transform: rotate and sort

Cyclic rotations

banana$
anana$b
nana$ba
ana$ban
na$bana
a$banan
$banana

Sorted rotations

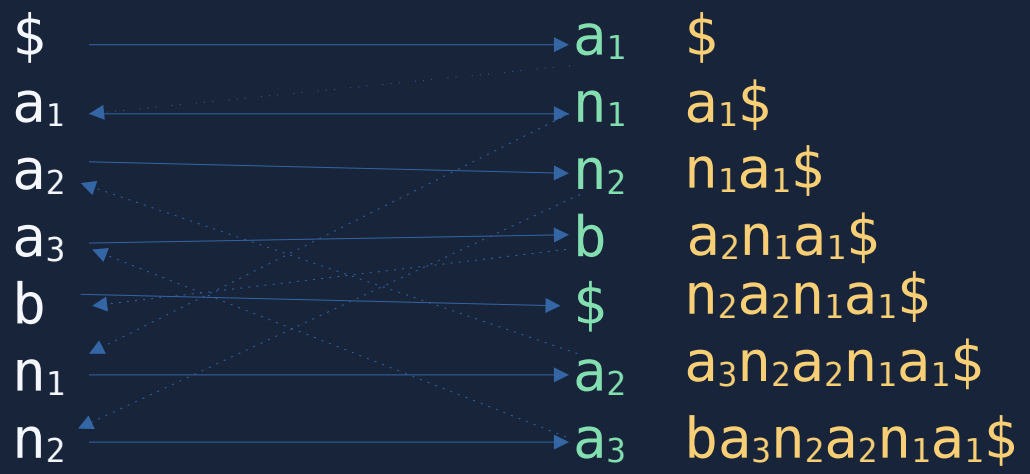
$banana
a$banan
ana$ban
anana$b
banana$
na$bana
nana$ba

The transformed string is the last column of the sorted rotations : annb$aa

Inverse: link last and first columns

Rebuild original text by prepending while you traverses the last to first mapping.

F is simply sorted L



We know that the last column was obtained from a sorted list of rotations, so we can obtain the first column by sorting the letters.

The last-to-first rule

Match the k-th occurrence of each character in L to the k-th occurrence of that same character in F.
This moves one character backward through the original text.

Three things to remember

1 **BWT is a lossless rearrangement algorithm**

… not compression!

2 **Contexts create clustering → runs of letters**

Certain letters tends to precede certain constructs “t before h”, “h before e”

3 **Fully reversible**

Match equal letters by their occurrence number in F and L.

`banana$` → `annb$aa` → `banana$`